EDA for WINWQUALITY dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Load Dataset

```
df = pd.read_csv("winequality-red.csv")
df.head()
```

Show hidden output

Check dataset information:

```
df.shape
df.info()
df.describe()
```

Show hidden output

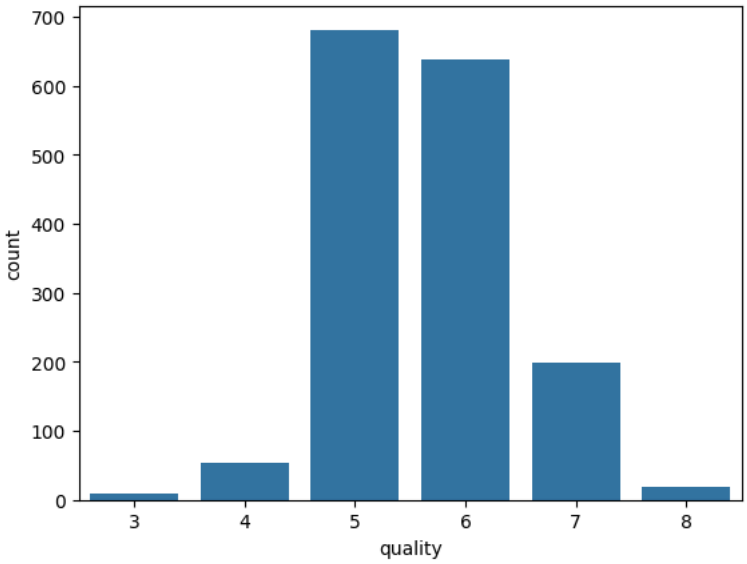
Exploratory Data Analysis (EDA)

Check Missing Values

```
df.isnull().sum()
```

Show hidden output

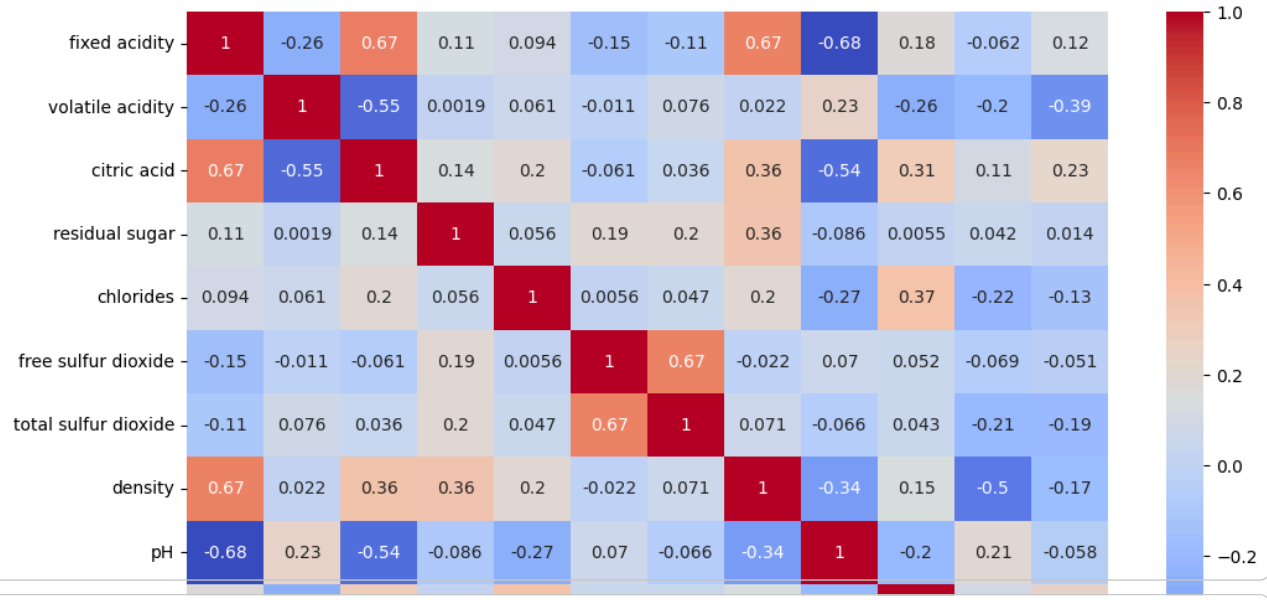
```
sns.countplot(x='quality', data=df)
plt.show()
```



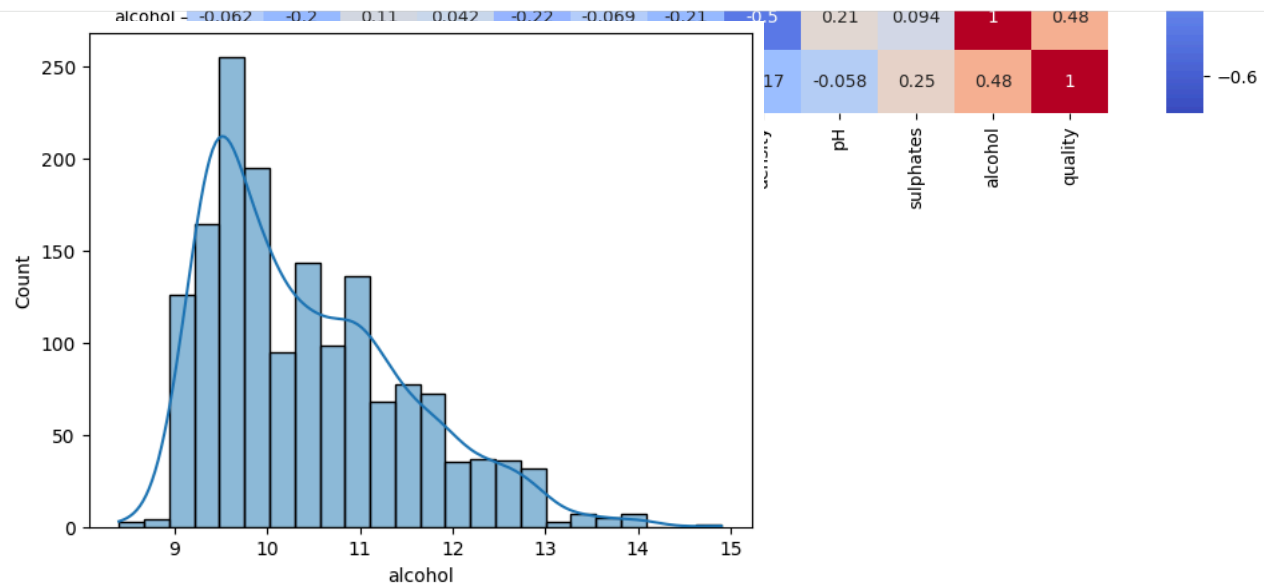
Distribution of Target Variable

- countplot
- Correlation Heatmap
- Bar Plot
- Box Plot
- Histogram

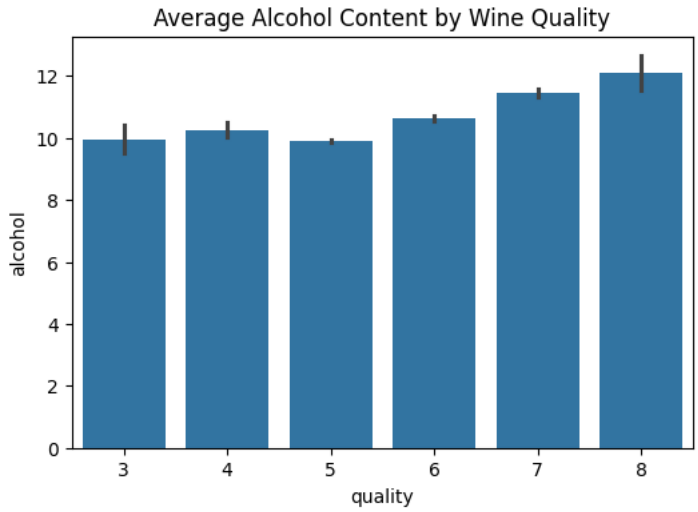
```
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```



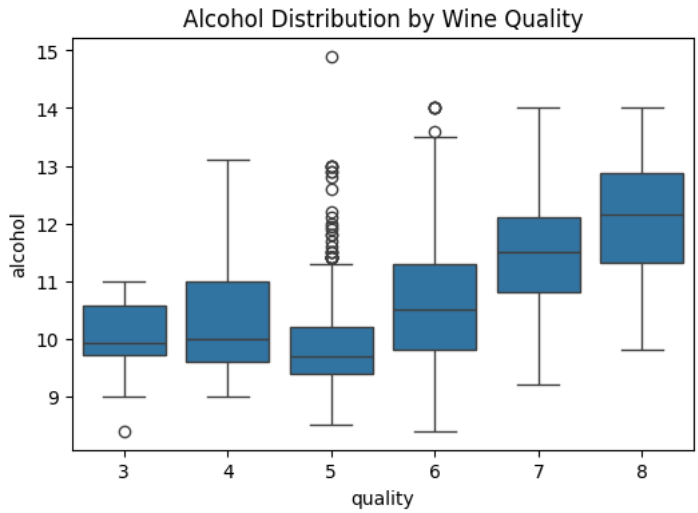

```
sns.histplot(df['alcohol'], kde=True)
plt.show()
```



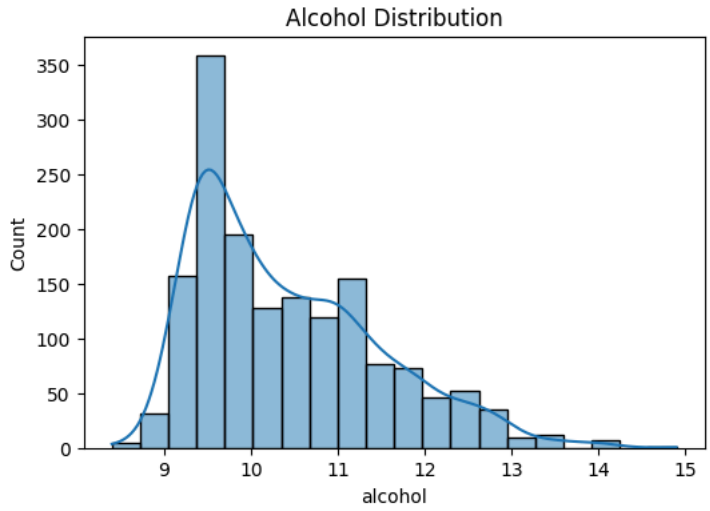

```
plt.figure(figsize=(6,4))
sns.barplot(x='quality', y='alcohol', data=df)
plt.title("Average Alcohol Content by Wine Quality")
plt.show()
```



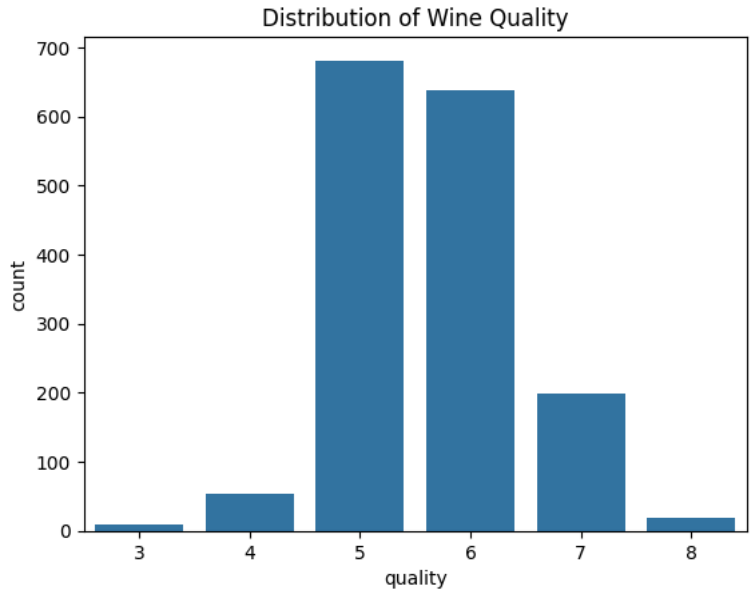

```
plt.figure(figsize=(6,4))
sns.boxplot(x='quality', y='alcohol', data=df)
plt.title("Alcohol Distribution by Wine Quality")
plt.show()
```




```
plt.figure(figsize=(6,4))
sns.histplot(df['alcohol'], bins=20, kde=True)
plt.title("Alcohol Distribution")
plt.show()
```

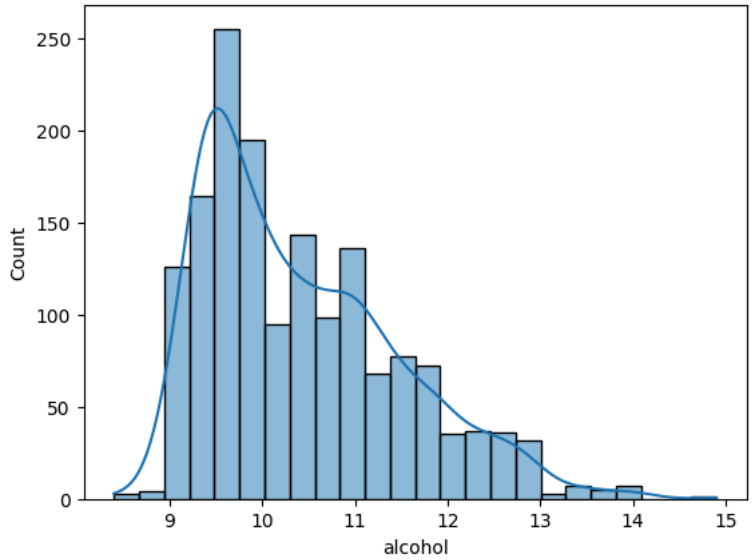



```
sns.countplot(x='quality', data=df)
plt.title("Distribution of Wine Quality")
plt.show()
```



Feature Distribution

```
sns.histplot(df['alcohol'], kde=True)
plt.show()
```



Data Preprocessing

Separate Features and Target

```
X = df.drop('quality', axis=1)
y = df['quality']
```

```
y = y.apply(lambda x: 1 if x >=6 else 0)
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
Start coding or generate with AI.
```

```
X_train.shape
```

```
(1279, 11)
```

```
X_test.shape
```

```
(320, 11)
```

Model Training

Logistic Regression

```
lr = LogisticRegression()
lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)
```

```
print("Logistic Regression Accuracy:",
      accuracy_score(y_test, y_pred_lr))
```

Show hidden output

Linear Regression

```
from sklearn.linear_model import LinearRegression
model= LinearRegression()


model.fit(X_train, y_train)
```

Show hidden output

```
y_pred = model.predict(X_test)
```

```
y_pred = (y_pred >= 0.5).astype(int)
print("Linear Regression Accuracy:",
      accuracy_score(y_test, y_pred))
```

```
Linear Regression Accuracy: 0.75
```

```
from sklearn.metrics import mean_squared_error, r2_score
```

```
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
```

```
Mean Squared Error: 0.25
```

R^2 Score

```
r2 = r2_score(y_test, y_pred)
print("R2 Score:", r2)
```